第九讲 - 异常处理 (编程练习)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



1 业务问题引入

2 从报错信息认识异常

3 捕获和处理异常

4 多种异常与完整处理流程

5 主动检查与抛出异常

6 异常传播与处理边界

7 总结与课后拓展

Python 程序设计基础

第8讲 文件读写 速记卡

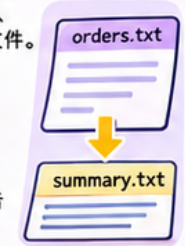
让程序学会读取数据、处理数据、保存结果！

Python
数据处理
电商实践

读取数据
处理数据
保存结果

1 为什么要学文件读写？

- 用一个简单的订单统计场景来说明：程序如何从文件读取数据、处理数据、并把结果保存成新文件。



- ✓ 订单数据需要长期保存
- ✓ 程序要读取 orders.txt
- ✓ 只统计“已完成”订单
- ✓ 计算订单数量和金额
- ✓ 生成新的 summary.txt 报告

订单数据（orders.txt）示例：

```
0001|杭州|360.00|已完成
0002|上海|640.00|已完成
0003|宁波|520.00|已退款
```

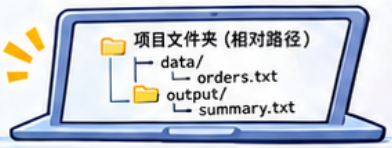
期望结果：

已完成订单 2笔，
已完成金额 1000.00元

2 文件与路径

- 文件 = 保存在电脑里的数据（如订单、报表、日志等）。
- 相对路径示例：data/orders.txt（项目下的 data 文件夹中的订单文件）。
- 输出路径示例：output/summary.txt（把统计结果保存到 output 文件夹）。
- 相对路径便于移动项目（换一台电脑也能用）。
- 可用 Path 检查文件是否存在。

```
from pathlib import Path
file_path = Path("data") / "orders.txt"
file_path.exists()  # 返回 True 或 False
```



安全打开与关闭文件

- ✓ 用 with open(...) 更安全、推荐使用。
- ✓ 离开代码块后，文件会自动关闭（不用再手动 file.close()）。
- ✓ 打开文件时需要说明：路径、模式、编码。

```
with open("data/orders.txt", "r",
          encoding="utf-8") as file:
    content = file.read()
```

常用的文件模式：

r	读取：打开文件并读取内容（最常用）。
w	重写：写入文件，会覆盖原有内容！
a	追加：在文件末尾追加内容，不会覆盖。

读取文本文件

- Python 提供多种读取文本的方式，常用方法：

读取方法	作用（通俗理解）
read()	一次读全部：把整个文件内容读成一个字符串。
readline()	读下一行：每次读取一行文本。
readlines()	得到行列表：把所有行读成一个列表。
for line in file	逐行处理：循环读取每一行（常用、推荐）。

订单记录通常适合逐行处理：一行就是一笔订单，便于解析和筛选。

```
for line in file:
    record = line.strip()  # 去掉行首尾的空白
    if not record:
        continue       # 跳过空行
```

5 文本清洗与数据转换

- 把从文件读到的“原始文本”，一步步变成可用的结构化数据（如订单号、城市、金额、状态）。

原始文本（一行订单） → strip() 去空白 → split("|") 按|分割 → 命名字段 更清晰 → float() 转金额（字符串→数字）

```
record = "0001|杭州|360.00|已完成"
fields = record.split("|")
order_id, city, amount_text, status = fields
amount = float(amount_text)
```

转换成更清晰的数据（字典示例）：

```
{"订单号": "0001",
 "城市": "杭州",
 "金额": 360.0,
 "状态": "已完成"}
```

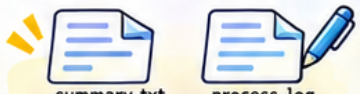


6 写入与追加结果

- ✓ write()：向文件写入一个字符串。
- ✓ print(..., file=file)：也能把内容写入文件。
- ✓ w 模式：重写文件（会覆盖原内容）。适合生成新的报告（如 summary.txt）。
- ✓ a 模式：追加内容（不覆盖）。适合记录日志（如 process.log）。

```
with open("output/summary.txt", "w",
          encoding="utf-8") as file:
    print("订单统计报告", file=file)
```

```
with open("output/process.log", "a",
          encoding="utf-8") as file:
    file.write("订单统计完成\n")
```



7 综合业务实践：订单统计报告

- 把所学知识串起来，完成一个完整的订单统计小项目！

业务流程（6 步）：

1. 读取订单文件
2. 解析每行订单
3. 只保留“已完成”订单
4. 汇总数量与金额
5. 写入 summary.txt
6. 重新读取并核验结果

关键函数（示例）：

- parse_order(line) → 解析一行订单，返回字典。
- read_orders(file_path) → 读取订单文件，得到订单列表。
- summarize_orders(orders) → 汇总“已完成”订单的数量和金额。
- write_report(file_path, summary) → 把统计结果写入文件。



运行结果（summary.txt）：

订单统计报告
已完成订单：2笔
已完成金额：1000.00元

学习提醒 / 易错点

- ✓ **当前工作目录不对，路径会报错：** 确保程序运行时的目录能找到你的文件。
- ✓ **编码通常用 UTF-8：** 如 encoding="utf-8"，可避免中文乱码。
- ✓ **w 会覆盖原内容，别乱用：** 需要保留原内容时，使用 a 模式追加。
- ✓ **文件读到的数据常常是字符串：** 如金额需要用 float() 转成数字后再计算。
- ✓ **写完文件后，最好重新读取检查结果：** 确认文件内容是否正确。
- ✓ **先想清楚：输入文件、处理规则、输出文件：** 理清流程，写代码更顺畅。



小提示：
多动手练一练，
从处理一份小数据开始，
你也能用 Python
完成实用的数据任务！

记住：文件读写不是背语法，而是把“外部数据 → 程序处理 → 结果保存”连成完整流程。

订单统计为什么没有完成

订单文件orders.txt

```
0001|杭州|360.00|已完成
0002|上海|六百四十|已完成
0003|宁波|520.00
0004|杭州|120.00|已完成
```

业务任务

读取全部订单，统计已完成订单的数量和金额，再生成一份文本报告。

```
orders = []

with open("data/orders.txt", "r", encoding="utf-8") as file:
  for line in file:
    order_id, city, amount_text, status = line.strip().split("|")
    orders.append({
      "订单号": order_id,
      "城市": city,
      "金额": float(amount_text),
      "状态": status
    })
..... # 生成文本报告的代码
```

程序运行结果：程序处理第二行时，无法把“六百四十”转换为浮点数，因此产生 `ValueError` 并停止运行。第三、四行没有处理，统计报告也没有生成。

不同错误需要不同处理策略

发生的问题	能否继续	合适的处理方式
订单文件不存在	不能	提示文件路径错误，停止本次统计
某一行金额格式错误	可以	记录行号，跳过当前订单，继续处理其他记录
某一行缺少字段	可以	记录错误原因，跳过当前订单
统计报告无法写入	不能	停止任务，并明确提示报告没有生成

异常处理需要回答的问题

- 程序在哪一步失败；
- 失败原因是什么；
- 当前任务能否安全继续；
- 用户需要看到哪些错误信息；
- 最终统计结果是否仍然可信。

本讲的核心任务

异常处理需要根据业务影响选择继续、跳过或停止，同时保留足够的错误信息，并检查程序是否产生了可信结果。

程序问题的三种表现

```python
# 1. 语法错误：缺少冒号
if status == "已完成"
  completed_count += 1

# 2. 运行时异常
amount = float("六百四十")

# 3. 逻辑错误：统计条件写反
if status == "已退款":
  completed_amount += amount
```

1. **语法错误**

   Python无法正确理解代码，程序不能正常开始执行。

2. **运行时异常**

   程序已经开始执行，但某项操作无法完成，正常流程被中断。

3. **逻辑错误**

   程序能够运行，也没有产生异常，但最终结果不符合业务规则。

对应的处理方式

语法错误需要修改代码；运行时异常需要判断是否捕获；逻辑错误需要通过测试、人工计算和结果核验发现。

异常处理的范围

`try/except` 主要处理运行过程中可能出现的异常。它不能自动修复语法错误，也不能保证业务计算结果正确。

Traceback中的关键信息

```
Traceback (most recent call last):
  File "order_report.py", line 18, in <module>
    amount = float(amount_text)
ValueError: could not convert string to float: '六百四十'
```

先看最后一行

`ValueError` 是异常类型。后面的英文信息说明Python无法把指定字符串转换为浮点数。

再看出错位置

错误发生在 `order_report.py` 第18行，具体操作是 `float(amount_text)` 。

Traceback的阅读顺序

1. 从最后一行确认异常类型和异常消息；
2. 向上查找自己编写的文件和代码行；
3. 检查参与运算的数据及其类型；
4. 判断应修改代码、修正数据还是捕获异常。

本课程中的常见异常

异常类型	典型操作	常见原因
`ValueError`	`float("六百四十")`	数据类型允许转换，但具体内容不符合要求
`TypeError`	`"100" + 20`	对不兼容的数据类型执行操作
`IndexError`	`orders[5]`	索引超出列表的有效范围
`KeyError`	`order["折扣"]`	字典中不存在指定的键
`FileNotFoundError`	打开订单文件	文件不存在，或者文件路径不正确
`ZeroDivisionError`	`sales / order_count`	除数的值为零

不需要背异常名称

Python会在Traceback最后一行给出异常类型。需要理解错误信息，并把它与当前执行的操作联系起来。

相同异常可能来自不同原因

例如 `ValueError` 既可能来自金额格式错误，也可能来自拆分字段后变量数量不匹配。异常类型只是定位问题的起点。

捕获一个预期异常

```python
amount_text = "六百四十"

try:
    amount = float(amount_text)
    print(f"订单金额: {amount:.2f}元")
except ValueError:
    print(f"金额格式错误: {amount_text}")

print("继续执行后面的程序")
```

执行顺序

1. Python先执行 `try` 代码块;
2. `float()` 产生 `ValueError` ;
3. `try` 中剩余代码停止执行;
4. Python转到匹配的 `except` 代码块;
5. 异常处理结束后，程序继续向下执行。

只捕获预期出现的异常

这里预期金额文本可能无法转换，因此捕获 `ValueError` 。其他类型的异常不会被这个 `except` 处理。

保留异常信息

```
line_number = 2
amount_text = "六百四十"

try:
  amount = float(amount_text)
except ValueError as error:
  print(
    f"第{line_number}行金额格式错误: "
    f"{amount_text}"
  )

  print(f"Python提供的信息: {error}")
```

异常对象

`as error` 把本次异常对象保存到变量 `error`。将它转换为字符串后，可以得到Python提供的错误消息。

补充业务上下文

Python知道数值转换失败，但不知道它来自订单文件的第几行。程序需要补充行号、字段名等业务信息。

一条有用的错误信息：错误信息应说明错误发生的位置、涉及的数据和失败原因。只输出“程序出错了”通常无法帮助用户定位问题。

缩小异常捕获范围 I

```
completed_amount = 0.0

with open("data/orders.txt", "r", encoding="utf-8") as file:
  for line_number, line in enumerate(file, start=1):
    fields = line.strip().split("|")
    amount_text = fields[2]

    try:
      amount = float(amount_text)
    except ValueError as error:
      print(
        f"第{line_number}行金额错误: "
        f"{error}"
      )
      continue

    completed_amount += amount

print(completed_amount)
```

缩小异常捕获范围 II

try代码块只保护转换操作

程序预期 `float(amount_text)` 可能失败，因此只把这一行放入 `try` 。后面的金额累计不被认为可能出现数据转换错误。

捕获后需要明确处理策略

这里使用 `continue` 跳过错误记录。如果错误会影响全部统计结果，则应停止任务，而不是继续计算。

避免隐藏错误

不要使用空的 `except:` 配合 `pass` 忽略所有问题。这会让程序看似正常运行，却可能遗漏数据或产生错误结果。

针对不同异常分别处理

```
file_path = "data/orders.txt"
try:
  with open(file_path, "r", encoding="utf-8") as file:
    content = file.read()
except FileNotFoundError:
  print(f"订单文件不存在：{file_path}")
except PermissionError:
  print(f"没有权限读取文件：{file_path}")
except UnicodeDecodeError as error:
  print(f"订单文件不是有效的UTF-8文本：{error}")
```

多个处理分支

Python从上到下检查 `except` 。找到第一个匹配的异常类型后，执行对应的处理代码（类似选择语句）。

不同问题需要不同提示

文件不存在时应检查路径，权限不足时应检查访问权限，编码错误时应确认文件的保存格式。

相同处理方式可以合并：如果多个异常采用完全相同的处理方式，可以写成 `except (FileNotFoundError, PermissionError) as error:` 。需要不同提示时，应使用独立的 `except` 分支。

异常处理的完整流程

```
file_path = "data/orders.txt"
try:
  with open(file_path, "r", encoding="utf-8") as file:
    content = file.read()
except FileNotFoundError:
  print(f"订单文件不存在：{file_path}")
except PermissionError:
  print(f"没有权限读取文件：{file_path}")
else:
  line_count = len(content.splitlines())
  print(f"读取成功，共{line_count}行")
finally:
  print("订单文件读取步骤结束")
```

代码块	执行条件	主要用途
`try`	首先执行	放置可能产生预期异常的代码
`except`	出现匹配异常	提示原因并执行对应处理策略
`else`	没有发生异常	处理读取成功后的业务逻辑
`finally`	无论是否异常	执行必须完成的结束或清理操作

用异常表达无效业务数据

```python
def calculate_discount(amount, discount_rate):
    if amount < 0:
        raise ValueError("订单金额不能为负数")
    if not 0 <= discount_rate <= 1:
        raise ValueError("折扣率必须在0和1之间")
    return amount * discount_rate
```

主动检查业务规则

`raise` 会立即结束当前函数，并产生指定类型的异常。这里假定数据类型没有问题，但数据值可能不符合业务规则，因此使用 `ValueError`。

编写清晰的异常信息

异常信息应直接说明违反了什么规则。尽量使用具体的异常类型，不要笼统地使用 `Exception`。

检查规则与处理策略相分离

```
orders = [{"订单号": "0001", "金额": 360.0, "折扣率": 0.10},
{"订单号": "0002", "金额": 640.0, "折扣率": 1.20}]

for order in orders:
  try:
    discount = calculate_discount(order["金额"], order["折扣率"])
  except ValueError as error:
    print(f"{order['订单号']}无法计算折扣: {error}")
    continue
  print(f"{order['订单号']}折扣金额: {discount:.2f}元")
```

各部分的职责

函数负责检查通用业务规则并抛出异常；调用者掌握订单号等上下文，负责记录问题，并决定跳过当前订单还是停止程序。

何时主动抛出异常

当数据使函数无法完成约定的任务时，可以主动抛出异常。如果当前代码没有处理该异常，它会继续向调用该函数的位置传播。

异常沿函数调用链传播

```
def parse_amount(text):
  return float(text)

def calculate_payable(order):
  amount = parse_amount(order["金额"])
  return amount * (1 - order["折扣率"])

order = {"订单号": "0002", "金额": "六百四十", "折扣率": 0.10}

try:
  payable = calculate_payable(order)
except ValueError as error:
  print(f"{order['订单号']}金额无效: {error}")
```

传播过程

`parse_amount()` 产生 `ValueError` 后，`calculate_payable()` 没有处理该异常，异常便继续传播到调用它的位置，最终由外层的 `except` 处理。如果始终没有匹配的处理代码，异常会传播到程序最外层，Python随后停止程序并显示完整的 `Traceback` 。

异常应该在哪一层处理

过早处理会隐藏问题

```
def parse_amount(text):
  try:
    return float(text)
  except ValueError:
    return 0.0
```

错误金额被替换为 `0.0`，调用者无法区分真实的零金额和无效数据。

外层掌握业务上下文

```
for order in orders:
  try:
    amount = parse_amount(order["金额"])
  except ValueError as error:
    print(f"{order['订单号']}: {error}")
    continue
```

在能够确定哪条订单出错，并能决定跳过、重试还是停止程序的位置捕获异常。外层知道订单号，也能决定记录问题并跳过当前订单。如果当前函数无法正确处理，就让异常继续传播。

记录异常后继续传播

```
def process_order(order):
  try:
    return calculate_payable(order)
  except ValueError:
    print(f"记录失败订单：{order['订单号']}")
    raise

try:
  payable = process_order(order)
except ValueError as error:
  print(f"本次跳过该订单：{error}")
```

三种处理选择

- 当前层能够恢复程序时，捕获并完成处理；
- 当前层只记录局部信息时，记录后使用 `raise` 继续传播；
- 当前层无法增加有效信息时，不必捕获，让异常自动传播。

不带参数的raise：在 `except` 中单独使用 `raise`，可以再次抛出当前异常，并保留原来的异常类型、信息和调用过程。

异常处理的完整思路

需要回答的问题	处理方法
发生了什么	从 `Traceback` 最后一行开始阅读，确认异常类型和错误信息。
在哪里发生	找到发生异常的代码行，并检查该行使用的数据。
是否能够预料	文件、用户输入和外部数据可能产生的异常，通常需要提前考虑。
在哪里处理	**在掌握足够业务信息并且能够决定处理策略的位置捕获异常。**
如何处理	根据业务影响选择提示、跳过当前数据、重试或者停止程序。
如何验证	分别测试正常数据和预期的异常情况，检查程序是否按设计继续运行。

本讲的核心原则

只捕获可以预料的具体异常，保留有助于定位问题的信息，让程序采取明确的后续行动。

课后实践：稳健的订单处理程序

实践任务

读取 `orders.txt` 中的订单，计算折扣金额和实付金额。将有效订单写入 `valid_orders.txt`，将无效订单及其错误原因写入 `error_orders.log`。

程序要求

- 文件不存在时，显示文件路径并停止本次处理；
- 金额无法转换时，记录错误并跳过当前订单；
- 折扣率不在0到1之间时，主动抛出 `ValueError`；
- 程序结束时，输出成功订单数和失败订单数；
- 不使用空的 `except`，也不使用 `except: pass`。

建议测试的数据

至少测试正常订单、中文金额、缺少字段、负数金额、无效折扣率和文件不存在。可以用AI补充测试数据，但自己要知道每种数据预期触发的处理流程。

课后拓展：保留异常的原始原因

```python
def parse_amount(order):
    try:
        return float(order["金额"])
    except ValueError as error:
        raise ValueError(f"订单{order['订单号']}的金额无效") from error
```

异常链

`raise ... from error` 抛出新的异常，并记录新异常与原异常之间的关系。`Traceback` 会同时显示原始转换错误和补充后的业务信息。

使用场景

底层函数通常只知道“字符串无法转换”，上层函数还知道订单号等业务信息。异常链可以补充这些信息，同时保留最初的错误原因。

课后拓展：自定义业务异常

```python
class InvalidOrderError(ValueError):
  pass

def validate_order(order):
  if order["金额"] < 0:
    raise InvalidOrderError(f"{order['订单号']}的金额不能为负数")

try:
  validate_order(order)
except InvalidOrderError as error:
  print(f"订单校验失败：{error}")
```

自定义异常的作用

`InvalidOrderError` 这个名字明确表示订单违反业务规则。调用者可以将它与Python已有的异常类型区分处理。

何时需要：写程序时，通常优先使用Python已有的异常类型。当项目需要为某类业务错误设计独立处理策略时，再考虑定义新的异常类型。

THE END